Chapter 6 | Worked solutions with Python

Training and Generating with a Small Transformer

All fifteen exercises are solved in order. The first calculations isolate one idea at a time; the later experiments connect learning, generation, checkpoint selection, and caching. A small color-reversal task keeps every target easy to inspect.

The main distinction is what changes: training updates weights; validation scores fixed weights; generation extends a chosen prefix. A KV cache reuses intermediate vectors for that fixed predictor. The saved experiments show why low loss with leaked answers and a high stopping rate do not establish correct generation.

Run this notebook inside `transformers-book` with the packages in `requirements-ch06.txt` and Jupyter. All data are generated locally. Run cells from top to bottom; the later exercises train small CPU models and may take a few minutes. No source files are modified. The optional leakage experiment temporarily patches both mask bindings and restores them on leaving its context.

The written PDF contains every explanation and the measured results. The notebook PDF additionally includes the calculations and their outputs. Re-running with a different numerical environment can change training trajectories and timings; the environment is printed below.

In [1]

```
from pathlib import Path
import sys, math, json, platform, random
from copy import deepcopy
from tempfile import TemporaryDirectory
from unittest.mock import patch
from contextlib import redirect_stdout
from io import StringIO
import torch
from torch import nn
from torch.nn import functional as F

for repo_root in (Path.cwd(), *Path.cwd().parents):
    if (repo_root / "examples/ch06_training/training.py").is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")
from examples.ch05_transformer import model as architecture
from examples.ch05_transformer.model import Transformer, build_masks
from examples.ch06_training import decoding
from examples.ch06_training.data import BOS, EOS, PAD, VOCAB, collate
from examples.ch06_training.training import (
    evaluate, token_loss, train_step, save_checkpoint, load_checkpoint,
)
torch.set_num_threads(1)
torch.use_deterministic_algorithms(True)
print("Python", platform.python_version(), "PyTorch", torch.__version__)
print("Device: CPU; threads:", torch.get_num_threads())
```

Output

```
Python 3.12.14 PyTorch 2.15.0
Device: CPU; threads: 1
```

1. Follow what changes

For training, the source and shifted reference answer produce logits. Labels grade those logits. Backpropagation computes how the loss depends on the parameters, and the optimizer changes them. Supplying an answer for grading does not itself change a weight.

Activity	Weights	Prefix comes from	Loss needed?
Training update	Updated by the optimizer	Reference answer	Yes, to compute gradients
Teacher-forced validation	Fixed	Reference answer	Yes, to report NLL
Cached generation	Fixed	BOS, then chosen tokens	No, to choose the next token

`zero_grad()` clears previously stored gradients. `backward()` calculates gradients; it does not apply them. `optimizer.step()` updates parameters using those gradients. `eval()` changes training-specific module behavior, such as dropout, and does not disable differentiation. `no_grad()` avoids recording the graph for differentiation and does not switch off dropout. Our model has no running BatchNorm statistics; its LayerNorm behaves the same way in training and evaluation.

The check below deliberately computes a backward pass in evaluation mode. This demonstrates that evaluation mode alone permits gradients. The entire parameter state is unchanged after backward, but at least one parameter changes after the optimizer step.

In [2]

```
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(23)
    activity_model = Transformer(9, 9, d_model=8, n_heads=2,
                                 d_ff=16, dropout=0.0).double().eval()
activity_batch = collate([(3, 4, 5)])
activity_optimizer = torch.optim.SGD(activity_model.parameters(), lr=0.01)
before = [p.detach().clone() for p in activity_model.parameters()]
activity_optimizer.zero_grad(set_to_none=True)
logits = activity_model(activity_batch.source, activity_batch.decoder_input)
total, count = token_loss(logits, activity_batch.target)
(total / count).backward()
assert all(torch.equal(p, b) for p, b in zip(activity_model.parameters(), before))
assert any(p.grad is not None and p.grad.abs().sum() > 0
           for p in activity_model.parameters())
print("After backward: gradients exist; all parameters unchanged.")
activity_optimizer.step()
assert any(not torch.equal(p, b) for p, b in zip(activity_model.parameters(), before))
print("After step: parameters changed.")
```

Output

```
After backward: gradients exist; all parameters unchanged.
After step: parameters changed.
```

2. Align a padded batch

Start by reversing each source's colors. For `red green blue` the target colors are `blue green red`. For `red blue` they are `blue red`. Place `BOS` before the colors in the decoder input and `EOS` after them in the labels. At position j, the decoder input is the token just before the label in the completed answer. Right-pad each array to its own widest row. Here the source width is 3 and both target arrays have width 4.

Example	Source IDs as tokens	Decoder input	Label row
1	`red green blue`	`BOS blue green red`	`blue green red EOS`
2	`red blue PAD`	`BOS blue red PAD`	`blue red EOS PAD`

A `1` marks a valid input or an evaluated label. Build the source and target input masks from their inputs. Build the loss-selection mask from the **labels**. The two target masks happen to agree because the shift replaces one boundary token with another, leaving the same number of valid slots in each row.

Example	Source input validity	Decoder input validity	Loss selection
1	`1 1 1`	`1 1 1 1`	`1 1 1 1`
2	`1 1 0`	`1 1 1 0`	`1 1 1 0`

Read across each row of the causal grid to find the keys allowed for one query. The triangle allows key k for query q when $k \leq q$. At position 2 of example 1, the current input is `blue` and the label is `green`. The diagonal lets the query read `blue`, which is already available. The decoder input `green` is one position later, outside that row's allowed keys.

Query position	Key 1	Key 2	Key 3	Key 4
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1

All target keys in example 1 are valid. In example 2 the key-validity mask removes column 4 from the triangle. Its padded query in row 4 can still read the first three keys. The loss-selection mask skips the output at that position because its label is `PAD`.

In [3]

```
import math

from examples.ch05_transformer.model import Transformer, build_masks
from examples.ch06_training.data import BOS, EOS, PAD, VOCAB, collate
from examples.ch06_training.run_experiment import loss_sanity_example

RED, GREEN, BLUE = (VOCAB.index(c) for c in ("red", "green", "blue"))
batch_2 = collate([(RED, GREEN, BLUE), (RED, BLUE)])
loss_selected_2 = batch_2.target != PAD

assert batch_2.source.tolist() == [
    [RED, GREEN, BLUE], [RED, BLUE, PAD],
]
assert batch_2.decoder_input.tolist() == [
    [BOS, BLUE, GREEN, RED], [BOS, BLUE, RED, PAD],
]
assert batch_2.target.tolist() == [
    [BLUE, GREEN, RED, EOS], [BLUE, RED, EOS, PAD],
]
assert batch_2.source_valid.int().tolist() == [[1, 1, 1], [1, 1, 0]]
assert batch_2.target_valid.int().tolist() == [[1, 1, 1, 1], [1, 1, 1, 0]]
assert loss_selected_2.int().tolist() == [[1, 1, 1, 1], [1, 1, 1, 0]]

print("Source IDs      ", batch_2.source.tolist())
print("Decoder input   ", batch_2.decoder_input.tolist())
print("Labels          ", batch_2.target.tolist())
print("Source valid    ", batch_2.source_valid.int().tolist())
print("Decoder valid   ", batch_2.target_valid.int().tolist())
print("Loss selection  ", loss_selected_2.int().tolist())
```

Output

```
Source IDs       [[3, 4, 5], [3, 5, 0]]
Decoder input    [[1, 5, 4, 3], [1, 5, 3, 0]]
Labels           [[5, 4, 3, 2], [5, 3, 2, 0]]
Source valid     [[1, 1, 1], [1, 1, 0]]
Decoder valid    [[1, 1, 1, 1], [1, 1, 1, 0]]
Loss selection   [[1, 1, 1, 1], [1, 1, 1, 0]]
```

In [4]

```
_, decoder_allowed_2 = build_masks(
    batch_2.source_valid, batch_2.target_valid
)
assert decoder_allowed_2.shape == (2, 1, 4, 4)
assert decoder_allowed_2[0, 0].int().tolist() == [
    [1, 0, 0, 0], [1, 1, 0, 0], [1, 1, 1, 0], [1, 1, 1, 1],
]
assert decoder_allowed_2[1, 0].int().tolist() == [
    [1, 0, 0, 0], [1, 1, 0, 0], [1, 1, 1, 0], [1, 1, 1, 0],
]
print("Allowed keys per query, example 2")
for row in decoder_allowed_2[1, 0].int().tolist():
    print(row)

sanity_2 = loss_sanity_example()
assert loss_selected_2.sum().item() == sanity_2["tokens"] == 7
assert math.isclose(sanity_2["loss_sum"], 7 * math.log(9))
assert math.isclose(sanity_2["perplexity"], 9)
print(f"Evaluated labels  {sanity_2['tokens']}")
print(f"Loss sum          {sanity_2['loss_sum']:.6f}")
print(f"Mean NLL          {sanity_2['nll']:.6f}")
print(f"Perplexity        {sanity_2['perplexity']:.6f}")
```

Output

```
Allowed keys per query, example 2
[1, 0, 0, 0]
[1, 1, 0, 0]
[1, 1, 1, 0]
[1, 1, 1, 0]
Evaluated labels  7
Loss sum          15.380572
Mean NLL          2.197225
Perplexity        9.000000
```

The count is $(3 + 1) + (2 + 1) = 7$ because each row contributes its colors and one `EOS` label. `BOS` is supplied as an input. The sanity check gives all nine vocabulary classes equal logits, so the probability of each correct token is $1/9$. Seven valid labels give a total loss $7 \ln 9$, a mean $\ln 9$, and perplexity $\exp(\ln 9) = 9$.

3. Calculate the signal

The three-class example is one vocabulary distribution: its entries sum to one, and the first class is correct. Its loss is $-\log(0.8) = 0.223144$. Differentiating the negative log of softmax gives

$$\frac{\partial \ell}{\partial z_v} = p_v - \mathbf{1}[v = y], \qquad \nabla_z \ell = (-0.20, 0.15, 0.05).$$

A small move opposite the gradient raises the correct logit and lowers the others. To isolate that effect, treat $z = \log(0.8, 0.15, 0.05)$ as trainable logits. One plain gradient-descent step of size 0.1 changes the correct probability to 0.805148 and the loss to 0.216729. This is a calculation on logits, not an Adam update of the Transformer's shared parameters.

In [5]

```
probabilities = torch.tensor([0.8, 0.15, 0.05], dtype=torch.float64)
logits = probabilities.log().requires_grad_()
loss = F.cross_entropy(logits[None], torch.tensor([0]))
loss.backward()
expected_gradient = probabilities - torch.tensor([1., 0., 0.])
torch.testing.assert_close(logits.grad, expected_gradient)
updated = logits.detach() - 0.1 * logits.grad
new_probability = updated.softmax(-1)[0].item()
new_loss = F.cross_entropy(updated[None], torch.tensor([0])).item()
assert new_probability > 0.8 and new_loss < loss.item()
print(f"Token loss: {loss.item():.6f}; logit gradient: {logits.grad.tolist()}")
print(f"After the step: p(correct)={new_probability:.6f}, loss={new_loss:.6f}")
```

Output

```
Token loss: 0.223144; logit gradient: [-0.20000000000000007, 0.15, 0.05000000000000001]
After the step: p(correct)=0.805148, loss=0.216729
```

The separate three-position example selects one correct-token probability from each of three predictions. Those selected values need not sum to one. Their losses are 0.223144, 0.693147, and 1.386294. The token mean is 0.767528, and its exponential is 2.154435. The last position contributes the largest penalty.

Perplexity is also $1/(0.8 \cdot 0.5 \cdot 0.25)^{1/3}$. It describes the geometric mean probability assigned to reference tokens, rather than counting wrong argmax choices.

In [6]

```
correct_probabilities = torch.tensor([.8, .5, .25], dtype=torch.float64)
penalties = -correct_probabilities.log()
mean_nll = penalties.mean().item()
perplexity = math.exp(mean_nll)
assert math.isclose(perplexity, correct_probabilities.prod().pow(-1/3).item())
print("Position losses:", [round(v, 6) for v in penalties.tolist()])
print(f"Mean NLL={mean_nll:.6f}; perplexity={perplexity:.6f}")
```

Output

```
Position losses: [0.223144, 0.693147, 1.386294]
Mean NLL=0.767528; perplexity=2.154435
```

4. Aggregate before dividing

Count the four evaluated tokens as four equal contributions. Three of them are in the first batch, and together they contribute 1.2 units of negative log-probability. The second batch contains one token with loss 1.0. The corpus mean is

$$\text{NLL} = \frac{1.2 + 1.0}{3 + 1} = \frac{2.2}{4} = 0.55, \qquad \text{perplexity} = e^{0.55} \approx 1.733253.$$

The two batch means are $1.2/3 = 0.4$ and $1.0/1 = 1.0$. Taking their unweighted average gives $(0.4+1.0)/2 = 0.7$, whose exponential is about 2.013753. This gives the single token in the second batch half the total influence, even though it accounts for one quarter of the labels. Weighting the batch means by the valid token counts recovers $(3 \cdot 0.4 + 1 \cdot 1.0)/4 = 0.55$.

We can check partitioning with a fixed float64 model in evaluation mode. The three sources below have lengths 3, 6, and 4. Their evaluated label counts are 4, 7, and 5, including `EOS`. A batch size of 1 separates all three examples. A batch size of 2 places the first two together and leaves the last one alone. Both calls use a generation cap of 8. The cap applies to the generated answers, whereas the NLL comes from the labeled predictions using their reference prefixes.

In [7]

```
from examples.ch06_training.training import evaluate, token_loss

def make_ch06_checks_model():
    with torch.random.fork_rng(devices=[]):
        torch.manual_seed(23)
        model = Transformer(
            len(VOCAB), len(VOCAB), d_model=8, n_heads=2, d_ff=16,
            n_encoder_layers=2, n_decoder_layers=2, dropout=0.0,
            max_length=32,
        )
    return model.double().eval()

model_4 = make_ch06_checks_model()
sequences_4 = [(3, 4, 5), (4, 4, 6, 6, 5, 3), (5, 3, 6, 4)]
cap_4 = 8
evaluation_4 = [
    evaluate(
        model_4, sequences_4, batch_size=size,
        generation_max_new_tokens=cap_4,
    )
    for size in (1, 2)
]
print("batch size   labels   corpus NLL       perplexity")
for size, result in zip((1, 2), evaluation_4):
    print(f"{size:10d}   {result['tokens']:6d}   "
          f"{result['nll']:.12f}   {result['perplexity']:.8f}")

assert evaluation_4[0]["tokens"] == evaluation_4[1]["tokens"] == 16
difference_4 = abs(evaluation_4[0]["nll"] - evaluation_4[1]["nll"])
assert difference_4 <= 1e-12
print(f"Absolute NLL difference  {difference_4:.3e}")
```

Output

```
batch size   labels   corpus NLL       perplexity
         1       16   2.419337529505   11.23841173
         2       16   2.419337529505   11.23841173
Absolute NLL difference  0.000e+00
```

In [8]

```
sums_4, counts_4 = [], []
with torch.no_grad():
    for subset in (sequences_4[:2], sequences_4[2:]):
        batch_4 = collate(subset)
        logits_4 = model_4(batch_4.source, batch_4.decoder_input)
        total_4, count_4 = token_loss(logits_4, batch_4.target)
        sums_4.append(total_4.item())
        counts_4.append(count_4.item())

mean_4 = sum(sums_4) / sum(counts_4)
unweighted_4 = sum(s / n for s, n in zip(sums_4, counts_4)) / 2
assert counts_4 == [11, 5]
assert math.isclose(mean_4, evaluation_4[1]["nll"], abs_tol=1e-12)
print("Batch loss sums  ", [round(s, 8) for s in sums_4])
print("Valid counts     ", counts_4)
print(f"Sum / total count         {mean_4:.12f}")
print(f"Unweighted batch average  {unweighted_4:.12f}")
```

Output

```
Batch loss sums   [24.04506736, 14.66433311]
Valid counts      [11, 5]
Sum / total count         2.419337529505
Unweighted batch average  2.559390918409
```

Batching changes where padding is stored and how the floating-point operations are grouped. Attention masks exclude padded keys, and loss selection skips padded labels. The same real labels contribute to the corpus mean in each partition. A tolerance of 10^{-12} accommodates the small arithmetic differences in this float64 check.

This untrained model has corpus NLL 2.419338 and perplexity 11.238412 under either partition. Perplexity can exceed the vocabulary size: nine is the uniform predictor's value, not a maximum. This model assigns lower geometric mean probability to the reference tokens than a uniform predictor does.

5. Limit a gradient

The vector $(3, 4)$ has Euclidean norm 5. With cap $c = 1$, multiply both entries by $\min(1, c/G) = 1/5$. The clipped vector is $(0.6, 0.8)$, whose norm is 1. For $(0.3, 0.4)$, the norm is 0.5, the scale factor is 1, and the vector stays unchanged.

Scaling every coordinate by the same positive number preserves direction. A zero gradient remains zero. Clipping limits the gradient supplied to Adam, whose actual parameter update also depends on its learning rate, moment estimates, bias correction, and numerical epsilon. The gradient cap does not impose that same bound on the parameter update. PyTorch uses a small stabilizer in its scale calculation, so its clipped result can be infinitesimally below the ideal norm.

In [9]

```
for values in ([3., 4.], [.3, .4]):
    gradient = torch.tensor(values, dtype=torch.float64)
    norm = gradient.norm().item()
    scale = min(1., 1. / norm)
    expected = gradient * scale
    parameter = nn.Parameter(torch.zeros_like(gradient))
    parameter.grad = gradient.clone()
    measured = nn.utils.clip_grad_norm_([parameter], 1.0)
    torch.testing.assert_close(parameter.grad, expected, atol=1e-6, rtol=1e-6)
    print(f"g={values}; norm={norm:g}; scale={scale:g}; "
          f"ideal clipped={expected.tolist()}; new norm={expected.norm():g}")
```

Output

```
g=[3.0, 4.0]; norm=5; scale=0.2; ideal clipped=[0.6000000000000001, 0.8]; new norm=1
g=[0.3, 0.4]; norm=0.5; scale=1; ideal clipped=[0.3, 0.4]; new norm=0.5
```

6. Trace the input, the choice, and the cache

One cache step consumes an input and returns scores for a new output. The newly selected output is not projected into K/V until the next call consumes it.

Position p	**Consumed input**	**Stored positions t per layer**	**Chosen output**
0	BOS	1	blue
1	blue	2	green
2	green	3	red
3	red	4	EOS

Immediately after choosing blue, only BOS has been consumed and cached. The final EOS ends this single-example generation and is never consumed. If a different row in the batch continues, the finished row can consume repeated EOS internally, but those later outputs are ignored.

Each decoder layer has its own input representation and learned projections, so it needs its own K/V. Old queries have already produced their outputs. A new position supplies a new query that reads the stored keys and combines their values. The check uses a supplied prefix with an untrained model: it verifies cache growth, not that this random model selects the illustrated colors.

In [10]

```
trace_model = deepcopy(activity_model).eval()
trace_batch = collate([(3, 4, 5)])
memory, allowed = decoding.encode_source(trace_model, trace_batch.source)
cache = decoding.CachedDecoder(trace_model, memory, allowed)
print("Position  Input    Stored K/V positions per layer")
for position, token in enumerate([BOS, 5, 4, 3]):
    cache.step(torch.tensor([[token]]))
    lengths = [key.shape[2] for key, value in cache.self_kv]
    assert lengths == [position + 1] * len(trace_model.decoder_layers)
    print(f"{position:8d}  {VOCAB[token]:7s}  {lengths}")
assert cache.position == 4
print("The selected EOS would stop generation before another step.")
```

Output

```
Position  Input    Stored K/V positions per layer
       0  <bos>    [1]
       1  blue     [2]
       2  green    [3]
       3  red      [4]
The selected EOS would stop generation before another step.
```

7. Account for memory

The factor two counts K and V. Multiply layers, examples, heads, cached positions, head width, and bytes per element:

$$2 \cdot 2 \cdot 3 \cdot 4 \cdot 20 \cdot 8 \cdot 4 = 30{,}720 \text{ bytes} = 30 \text{ KiB}.$$

Static cross-attention K/V uses source length $S = 6$ instead of $t = 20$: $9{,}216$ bytes, or 9 KiB. Together these tensors need $39{,}936$ bytes, or 39 KiB. Here $t = 20$ is a count: the last consumed position has index 19.

This excludes model parameters, encoder output memory, temporary attention scores and activations, allocator overhead, and any token-ID buffers. The simple implementation grows caches with concatenation, which can temporarily hold both old and new storage. The formula counts the final K/V tensors, not peak process memory.

In [11]

```
layers, batch_size, heads, width, stored, source = 2, 3, 4, 8, 20, 6
bytes_per_element = torch.empty((), dtype=torch.float32).element_size()
self_bytes = 2 * layers * batch_size * heads * stored * width * bytes_per_element
cross_bytes = 2 * layers * batch_size * heads * source * width * bytes_per_element
for name, size in [("Self", self_bytes), ("Cross", cross_bytes),
                   ("Total", self_bytes + cross_bytes)]:
    print(f"{name}: {size:,} bytes = {size / 1024:g} KiB")
assert (self_bytes, cross_bytes) == (30720, 9216)
```

Output

```
Self: 30,720 bytes = 30 KiB
Cross: 9,216 bytes = 9 KiB
Total: 39,936 bytes = 39 KiB
```

8. Test the stopping rule

Use the stop tests in `tests/test_training.py` as a model. Script the argmax choice at each step by giving that vocabulary class a large score and giving the other classes small scores. We can then check the stopping behavior without relying on a trained model to produce the particular token we need.

Predict the returned arrays before running the checks. `EOS` has ID 2, `PAD` has ID 0, and `green` has ID 4. The initial `BOS` is an input and is absent from the returned array.

Script	Expected tokens	Expected IDs	Decode calls
One row, immediate ending	`[[EOS]]`	`[[2]]`	1
One row, green through the cap	`[[green, green, green]]`	`[[4,4,4]]`	3
Two rows, one ends first	`[[EOS,PAD,PAD], [green,green,EOS]]`	`[[2,0,0], [4,4,2]]`	3

Call `greedy_decode` with `cached=False` so it calls the patched `decoding.decode_prefix`. The stub receives the accumulated prefix. Subtract one from its width to get the zero-based generation step because its first slot contains `BOS`. Fill only the last logits row, which is the row the generation loop uses for its next choice.

In [12]

```
from unittest.mock import patch
from examples.ch06_training import decoding

def scripted_prefix(schedule):
    def stub(model, prefix, memory, source_allowed):
        step = prefix.shape[1] - 1
        chosen = prefix.new_tensor(schedule[step]).unsqueeze(-1)
        logits = memory.new_full((*prefix.shape, len(VOCAB)), -100.0)
        logits[:, -1].scatter_(1, chosen, 10.0)
        return logits
    return stub

scripts_8 = (
    ("Immediate EOS", [[EOS]], [[EOS]]),
    ("Green to cap", [[GREEN], [GREEN], [GREEN]],
     [[GREEN, GREEN, GREEN]]),
    ("Mixed", [[EOS, GREEN], [EOS, GREEN], [EOS, EOS]],
     [[EOS, PAD, PAD], [GREEN, GREEN, EOS]]),
)
```

In [13]

```
model_8 = make_ch06_checks_model()
for name, script, expected in scripts_8:
    source_8 = batch_2.source[:len(expected)]
    with patch.object(
        decoding, "decode_prefix", side_effect=scripted_prefix(script)
    ) as traced:
        generated_8 = decoding.greedy_decode(
            model_8, source_8, max_new_tokens=3, cached=False,
        )
    assert generated_8.tolist() == expected
    assert traced.call_count == len(script)
    print(f"{name:15s}  {generated_8.tolist()}  calls={traced.call_count}")

    if name == "Mixed":
        finished_inputs_8 = [
            call.args[1][0, -1].item() for call in traced.call_args_list
        ]
        assert finished_inputs_8 == [BOS, EOS, EOS]
        print("Row 1 inputs during the three calls:", finished_inputs_8)
```

Output

```
Immediate EOS    [[2]]  calls=1
Green to cap     [[4, 4, 4]]  calls=3
Mixed            [[2, 0, 0], [4, 4, 2]]  calls=3
Row 1 inputs during the three calls: [1, 2, 2]
```

The generation loop stops when all rows have emitted `EOS` or when the cap has been reached. The second case reaches the cap with three colors and no ending, so its termination flag is false. In the mixed case, the first row emits `EOS` on the first call. The loop keeps calling the decoder while the second row is active. The completed row receives `EOS` as input during those calls, while the output stores `PAD` after the first `EOS` to keep the array rectangular.

9. Overfit before scaling

Choose four source sequences and keep presenting the same batch. For example, the answer to *red green blue* is *blue green red*, followed by EOS, the end-of-sequence token. The exercise isolates the link between decoder inputs and prediction targets. A correctly wired model should be able to memorize a few examples, and we can check that memorization by asking it to generate the answers.

At each decoder position, **teacher forcing** supplies the previous reference token as input. BOS, the beginning-of-sequence token, occupies the first position. The model must predict the next token in the row.

Decoder position	**1**	**2**	**3**	**4**
Shifted input	BOS	blue	green	red
Label to predict	blue	green	red	EOS
Deliberately unshifted input	blue	green	red	EOS

In the unshifted experiment, the current input reveals its own label. The causal mask permits the diagonal, so the current position can use that input. The decoder's residual paths also carry information from the current input embedding. Copying it can earn a low training loss with little need to learn the reversal.

The two runs begin from fresh models with the same seed and default initialization. Each uses width 16, two heads, FFN width 32, one layer per stack, zero dropout, and Adam with rate `0.01`. Each update uses all four

examples. The generation cap is six, enough to emit four colors and EOS with one position to spare. The scores and generation checks here use the four training examples.

In [14]

```
from examples.ch05_transformer.model import Transformer
from examples.ch06_training.data import Batch, VOCAB, collate, words
from examples.ch06_training.decoding import greedy_decode
from examples.ch06_training.training import (
    exact_matches, token_loss, train_step,
)

sources_overfit = [
    (3, 4, 5), (6, 5, 3), (7, 4, 8, 6), (8, 6, 5, 4),
]
shifted = collate(sources_overfit)
unshifted = Batch(
    shifted.source, shifted.target.clone(), shifted.target,
)
for source, target in zip(shifted.source, shifted.target):
    print(f"{words(source):27} -> {words(target, True)}")
```

Output

```
red green blue              -> blue green red <eos>
yellow blue red             -> red blue yellow <eos>
white green black yellow    -> yellow black green white <eos>
black yellow blue green     -> green blue yellow black <eos>
```

`collate` produces the shifted inputs and labels. The experimental batch copies the labels into the decoder input and keeps the same sources and labels. Its right-padding pattern also stays the same. Both models use the canonical `train_step`, including its gradient norm cap. At each checkpoint, switch to evaluation mode and recompute mean token loss with the input arrangement used in that run. Then start greedy decoding from BOS without supplying the reference sequence.

In [15]

```
def overfit_checkpoint(model, batch):
    model.eval()
    with torch.no_grad():
        logits = model(batch.source, batch.decoder_input)
        total, count = token_loss(logits, batch.target)
    generated = greedy_decode(
        model, batch.source, max_new_tokens=6, cached=False,
    )
    return {
        "nll": (total / count).item(),
        "matches": exact_matches(generated, batch.target),
        "rows": [words(row, include_special=True) for row in generated],
    }


def fit_four_examples(batch):
    with torch.random.fork_rng(devices=[]):
        torch.manual_seed(2026)
        model = Transformer(
            len(VOCAB), len(VOCAB), d_model=16, n_heads=2, d_ff=32,
            n_encoder_layers=1, n_decoder_layers=1, dropout=0.0,
        )
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    snapshots = {0: overfit_checkpoint(model, batch)}
    for step in range(1, 201):
        train_step(model, optimizer, batch)
        if step in (20, 75, 200):
            snapshots[step] = overfit_checkpoint(model, batch)
    return snapshots
```

In [16]

```
torch.set_num_threads(1)
shifted_trace = fit_four_examples(shifted)
unshifted_trace = fit_four_examples(unshifted)

assert shifted_trace[0]["rows"] == unshifted_trace[0]["rows"]
print("Update  Shifted NLL  Exact     Unshifted NLL  Exact")
for step, correct in shifted_trace.items():
    leaky = unshifted_trace[step]
    print(
        f"{step:>6}  {correct['nll']:>11.6f}  {correct['matches']}/4"
        f"       {leaky['nll']:>13.6f}  {leaky['matches']}/4"
    )

for label, trace in (
    ("Shifted", shifted_trace), ("Unshifted", unshifted_trace),
):
    for step in (20, 200):
        print(f"\n{label}, generated answers at update {step}")
        for row in trace[step]["rows"]:
            print(f"  {row}")
```

Output

```
Update  Shifted NLL  Exact     Unshifted NLL  Exact
     0     2.365434  0/4            2.315844  0/4
    20     0.219827  4/4            0.083590  0/4
    75     0.003178  4/4            0.003109  0/4
   200     0.000894  4/4            0.000953  0/4

Shifted, generated answers at update 20
  blue green red <eos>
  red blue yellow <eos>
  yellow black green white <eos>
  green blue yellow black <eos>

Shifted, generated answers at update 200
  blue green red <eos>
  red blue yellow <eos>
  yellow black green white <eos>
  green blue yellow black <eos>

Unshifted, generated answers at update 20
  green green green green green green
  green green green green green green
  green green green green green green
  green green green green green green

Unshifted, generated answers at update 200
  green green green green green green
  green green green green green green
  green green green green green green
  green green green green green green
```

Both models start with the same weights, so they generate the same answers before training. Their initial losses differ because their scoring inputs differ. The measured checkpoints make the failure clear:

Update	Shifted NLL	Exact answers	Unshifted NLL	Exact answers
0	2.365434	0/4	2.315844	0/4
20	0.219827	4/4	0.083590	0/4
75	0.003178	4/4	0.003109	0/4
200	0.000894	4/4	0.000953	0/4

By update 20, the correctly shifted model generates all four reversals and EOS. The unshifted model has a lower loss at that checkpoint but generates six copies of green for every source, reaching the cap without EOS. At update 200 both losses are about 0.001; the broken model still generates no exact answers. A low loss is evidence about the task the code actually grades. Here that task revealed the answer in the input. The canonical implementation is unchanged, and later exercises construct correctly shifted batches with `collate`.

10. Restore the same predictor

A checkpoint is a saved snapshot, not a live view of a model that keeps training. Save a float64 model, record logits for a fixed padded batch, and then deliberately change its live vocabulary bias. Loading must recover the original scores rather than these altered scores.

The checkpoint keeps the architecture, vocabulary order, weights, parameter dtype, experiment settings, selected update, and validation metadata. The loader reconstructs the architecture, checks the vocabulary, restores weights

at their saved precision, and enables evaluation mode. No KV cache is needed to restore this predictor: a fresh request can recompute it.

To resume the same Adam trajectory, additionally preserve optimizer moments and update counters, random-generator states, and sampler progress. A scheduler or mixed-precision scaler also needs its state when used. The example's inference checkpoint does not promise exact training resumption.

In [17]

```
restore_config = dict(source_vocab_size=9, target_vocab_size=9,
                      d_model=8, n_heads=2, d_ff=16, n_encoder_layers=2,
                      n_decoder_layers=2, dropout=0., max_length=32)
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(23)
    live = Transformer(**restore_config).double().eval()
restore_batch = collate([(3, 4, 5, 6), (6, 4, 3)])
with torch.no_grad():
    saved_logits = live(restore_batch.source, restore_batch.decoder_input).clone()
with TemporaryDirectory(prefix="ch06-restore-") as directory:
    checkpoint_path = Path(directory) / "predictor.pt"
    save_checkpoint(checkpoint_path, live, restore_config, {"seed": 23}, 7,
                    {"nll": .25})
    with torch.no_grad():
        live.vocabulary_projection.bias.add_(1.)
        changed = live(restore_batch.source, restore_batch.decoder_input)
    assert not torch.equal(changed, saved_logits)
    restored, metadata = load_checkpoint(checkpoint_path)
with torch.no_grad():
    recovered = restored(restore_batch.source, restore_batch.decoder_input)
torch.testing.assert_close(recovered, saved_logits, atol=1e-12, rtol=1e-12)
assert next(restored.parameters()).dtype == torch.float64
assert not restored.training and metadata["step"] == 7
print("Restored logits match; dtype=float64; evaluation mode; update=7.")
```

Output

```
Restored logits match; dtype=float64; evaluation mode; update=7.
```

11. Probe the limits of the task

A model may learn to reverse unfamiliar arrangements of familiar lengths, yet struggle when asked to reverse longer sequences. Keep the model fixed and measure those two abilities separately. First use the chapter's test split, whose lengths 3 through 6 occur in training. Then use a new test split of lengths 7 and 8. The colors, target format, and evaluation rules are shared.

To make the checkpoint available on a fresh clone, rerun the chapter's reference recipe in a temporary directory. It uses seed 2026 for the model, batch sampling, and data generation. The model has width 64, four heads, feed-forward width 128, two layers in each stack, and zero dropout. The recipe initializes the matrices with Xavier uniform, uses Adam with learning rate 0.001 and clips the gradient norm at 1. It performs 2,000 updates with batches of 64 and checks validation NLL every 100 updates, including the initial checkpoint at update zero. The lowest validation token NLL selects the saved weights.

The recorded run reports the elapsed time on one CPU thread. Training is seeded within the recorded environment. Results can vary across PyTorch versions, including the selected update. After reloading the checkpoint, every comparison below uses those same weights in evaluation mode.

In [18]

```
from contextlib import redirect_stdout
from io import StringIO
from tempfile import TemporaryDirectory

from examples.ch06_training.run_experiment import (
    run as run_reference_11,
    split_fingerprint as fingerprint_11,
)
from examples.ch06_training.training import (
    evaluate as evaluate_11,
    load_checkpoint as load_checkpoint_11,
)

with TemporaryDirectory(prefix="ch06-ex8-") as directory_11:
    with redirect_stdout(StringIO()):
        reference_11 = run_reference_11(
            directory_11, steps=2000, seed=2026, data_seed=2026,
            batch_size=64, validation_every=100,
        )
    model_11, checkpoint_11 = load_checkpoint_11(
        Path(directory_11) / "best.pt"
    )

print("Selected update", checkpoint_11["step"])
print("Validation NLL", f'{checkpoint_11["validation"]["nll"]:.8f}')
print("PyTorch", torch.__version__)
print("Parameter dtype", next(model_11.parameters()).dtype)
print("CPU threads", torch.get_num_threads())
print("Training and evaluation seconds",
      f'{reference_11["elapsed_seconds"]:.1f}')
reference_record = repo_root / "output/ch06-solutions/reference-metrics.json"
reference_record.parent.mkdir(parents=True, exist_ok=True)
reference_record.write_text(json.dumps(reference_11, indent=2) + "\n")
```

Output

```
Selected update 2000
Validation NLL 0.00049957
PyTorch 2.15.0
Parameter dtype torch.float32
CPU threads 1
Training and evaluation seconds 13.4
```

Reconstruct the original splits from the checkpoint's data seed and verify their stored fingerprints. Create the longer split with the exercise's requested sizes and use its 64 test sequences. Its training and validation portions are simply part of the split construction. We use neither for fitting or selection. There are 16 original test sequences per length and 32 longer test sequences per length.

Each evaluation call uses the same cap of 10 generated tokens. A source with eight colors needs eight output colors plus EOS, so that cap gives it room to finish. Using the same cap keeps the stopping allowance from changing across groups.

Token NLL averages $-\ln p$ over valid labels, including EOS, under the reference prefix. Token accuracy also uses those prefixes. In free-running greedy decoding, each selected token instead supplies the next decoder input. Exact match requires every target color in order and the final EOS. The EOS count only measures whether decoding ended by the cap.

In [19]

```
from examples.ch06_training.data import make_splits as make_splits_11

record_11 = checkpoint_11["experiment"]
seed_11 = record_11["data_seed"]
original_11 = make_splits_11(seed=seed_11)
for name_11, split_11 in zip(
    ("train", "validation", "test"), original_11
):
    assert fingerprint_11(split_11) == record_11["split_sha256"][name_11]
test_11 = original_11[2]
_, _, longer_test_11 = make_splits_11(
    seed=seed_11, min_length=7, max_length=8,
    n_train=8, n_validation=8, n_test=64,
)
cap_11 = 10

def report_lengths_11(model, sequences):
    def report(label, group):
        metrics = evaluate_11(
            model, group, generation_max_new_tokens=cap_11
        )
        size = metrics["sequences"]
        hits = round(metrics["exact_match"] * size)
        eos = round(metrics["termination_rate"] * size)
        print(
            f'{label:>6} {size:5d} {metrics["tokens"]:6d} '
            f'{metrics["nll"]:9.6f} '
            f'{metrics["token_accuracy"]:9.2%} '
            f'{hits:4d}/{size:<3d} {eos:3d}/{size:<3d}'
        )

    print("Length  Seq. Tokens       NLL  Tok. acc.     Exact     EOS")
    for length in sorted(set(map(len, sequences))):
        group = [source for source in sequences if len(source) == length]
        report(length, group)
    report("all", sequences)

print("Held-out lengths seen in training")
report_lengths_11(model_11, test_11)
print("\nHeld-out lengths beyond training")
report_lengths_11(model_11, longer_test_11)
```

Output

```
Held-out lengths seen in training
Length  Seq. Tokens       NLL  Tok. acc.     Exact     EOS
     3    16     64  0.004951   100.00%   16/16   16/16
     4    16     80  0.000282   100.00%   16/16   16/16
     5    16     96  0.000101   100.00%   16/16   16/16
     6    16    112  0.000362   100.00%   16/16   16/16
   all    64    352  0.001107   100.00%   64/64   64/64

Held-out lengths beyond training
Length  Seq. Tokens       NLL  Tok. acc.     Exact     EOS
     7    32    256  1.545827    72.27%    0/32   32/32
     8    32    288  5.176935    34.72%    0/32   32/32
   all    64    544  3.468178    52.39%    0/64   64/64
```

To see which errors contribute to the exact-match gap, decode each test batch with the same cap. Compare the emitted colors before the first EOS with the reversed source. In the examples below, a substitution is a wrong color at a position when the predicted and expected color sequences have equal length. That rule avoids calling

every shifted position a substitution when an output ends early. An excess repetition means a color appears at least twice in the generated sequence and more often than in the reference. A legitimate repeat can be part of the correct answer.

Check whether EOS arrives before or after the required number of colors, or never arrives within the cap. The counters can overlap. For each kind of error that occurs, show its first example in test order and keep EOS visible in the output.

In [20]

```
from collections import Counter

from examples.ch06_training.data import (
    EOS as EOS_11,
    collate as collate_11,
    words as words_11,
)
from examples.ch06_training.decoding import (
    greedy_decode as greedy_decode_11,
)

def trace_errors_11(model, sequences):
    generated = greedy_decode_11(
        model, collate_11(sequences).source, max_new_tokens=cap_11
    )
    counts = Counter()
    samples = {}
    for source, row in zip(sequences, generated.tolist()):
        expected = list(reversed(source))
        ended = EOS_11 in row
        colors = row[:row.index(EOS_11)] if ended else row
        difference = len(colors) - len(expected)
        excess_repeat = any(
            copies > max(1, expected.count(color))
            for color, copies in Counter(colors).items()
        )
        errors = {
            "Substitution at matching length": (
                difference == 0 and colors != expected
            ),
            "Excess repetition": excess_repeat,
            "Early EOS": ended and difference < 0,
            "Late EOS": ended and difference > 0,
            "No EOS by cap": not ended,
        }
        for kind, occurred in errors.items():
            counts[kind] += int(occurred)
            if occurred:
                samples.setdefault(kind, (source, expected, row))
    return counts, samples
```

In [21]

```
for name_11, sequences_11 in (
    ("In range", test_11), ("Extrapolation", longer_test_11)
):
    print(name_11)
    counts_11, samples_11 = trace_errors_11(model_11, sequences_11)
    for kind_11, count_11 in counts_11.items():
        print(f"{kind_11}: {count_11}/{len(sequences_11)}")
        if count_11:
            source_11, expected_11, row_11 = samples_11[kind_11]
            print("  Source   ", words_11(source_11))
            print("  Reference", words_11(expected_11 + [EOS_11], True))
            print("  Generated", words_11(row_11, True))
    print()
```

Output

```
In range
Substitution at matching length: 0/64
Excess repetition: 0/64
Early EOS: 0/64
Late EOS: 0/64
No EOS by cap: 0/64

Extrapolation
Substitution at matching length: 10/64
  Source    black red green black blue yellow blue
  Reference blue yellow blue black green red black <eos>
  Generated yellow blue black black green red black <eos>
Excess repetition: 22/64
  Source    black red green black blue yellow blue
  Reference blue yellow blue black green red black <eos>
  Generated yellow blue black black green red black <eos>
Early EOS: 54/64
  Source    yellow blue black red red red white
  Reference white red red red black blue yellow <eos>
  Generated red red red black black blue <eos>
Late EOS: 0/64
No EOS by cap: 0/64
```

The selected checkpoint is update 2000, with validation NLL 0.00049957. Its training history matches the separately recorded chapter run in this environment. The following measurements all use the same frozen checkpoint and generation cap 10:

Source length	Token NLL	Token accuracy	Exact answers	Ended by cap
3	0.004951	100.00%	16/16	16/16
4	0.000282	100.00%	16/16	16/16
5	0.000101	100.00%	16/16	16/16
6	0.000362	100.00%	16/16	16/16
7	1.545827	72.27%	0/32	32/32
8	5.176935	34.72%	0/32	32/32

The aggregate in-range NLL is 0.001107 over 352 labels; all 64 generated answers are exact. The aggregate extrapolation NLL is 3.468178 over 544 labels, with 52.39% teacher-forced token accuracy and no exact answers. These aggregates use label counts, not an unweighted average of the per-length means.

Every longer answer ends, but 54 of 64 end early. There are ten wrong answers of the correct length and 22 cases of excess repetition. These error categories overlap. For example, the source `black red green black`

`blue yellow blue` should produce `blue yellow blue black green red black EOS`; the model produces `yellow blue black black green red black EOS`. It has the correct output length but overuses black and loses one blue.

For `yellow blue black red red red white`, the output is `red red red black black blue EOS`: six colors instead of seven. This row has both excess repetition and early EOS. Perfect in-range test performance therefore does not establish reversal at unseen lengths. A 100% termination rate only says that an end token appeared by the cap.

12. Separate validation from testing (optional experiment)

Suppose two learning rates produce different trajectories. We need a way to select both a rate and an update before looking at the test set. Use the chapter's training, validation, and test splits with data seed 2026. Train on the training split only and rank checkpoints by validation token NLL. The test split supplies one final assessment of the selected checkpoint.

To keep this comparison quick, train a smaller model with width 32, four heads, feed-forward width 64, one encoder layer, one decoder layer, and dropout zero. Both runs start from copies of the same Xavier-initialized model. Both use Adam with default betas, batch size 32, the same seeded sequence of training batches, and gradient norm cap 1 through `train_step`. Only the learning rate differs.

Compare 0.001 with 0.004 over 240 updates each. Record validation at updates 0, 60, 120, 180, and 240 with cap 10. Include update zero as a valid checkpoint. Select the smallest validation NLL across both runs. If scores tie, prefer the earlier update, then the smaller learning rate. The rule and evaluation schedule are fixed before training.

In [22]

```
from copy import deepcopy
import random

from examples.ch05_transformer.model import (
    Transformer as Transformer_12,
)
from examples.ch06_training.data import (
    VOCAB as VOCAB_12,
    collate as collate_12,
    make_splits as make_splits_12,
)
from examples.ch06_training.training import (
    evaluate as evaluate_12,
    train_step as train_step_12,
)

train_12, validation_12, test_12 = make_splits_12(seed=2026)
torch.set_num_threads(1)
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(2026)
    initial_12 = Transformer_12(
        len(VOCAB_12), len(VOCAB_12), d_model=32, n_heads=4,
        d_ff=64, n_encoder_layers=1, n_decoder_layers=1,
        dropout=0.0, max_length=64,
    )
    for parameter in initial_12.parameters():
        if parameter.ndim > 1:
            nn.init.xavier_uniform_(parameter)

cap_12 = 10
rates_12 = (0.001, 0.004)
baseline_12 = evaluate_12(
    initial_12, validation_12, generation_max_new_tokens=cap_12
)
```

Each rate receives a fresh optimizer, so Adam’s state starts empty both times. Reset the training sampler to the same seed before each run. A validation result has no effect on the next training batch. Copy the winning model when it improves on the best candidate. Otherwise, further updates would overwrite the parameters of the checkpoint we meant to retain. The loop never calls the test set.

In [23]

```
best_key_12 = (baseline_12["nll"], 0, min(rates_12))
winner_12 = deepcopy(initial_12)
history_12 = [(rate, 0, baseline_12) for rate in rates_12]

for rate_12 in rates_12:
    candidate_12 = deepcopy(initial_12)
    optimizer_12 = torch.optim.Adam(candidate_12.parameters(), lr=rate_12)
    sampler_12 = random.Random(2026)
    for update_12 in range(1, 241):
        batch_12 = collate_12(sampler_12.sample(train_12, 32))
        train_step_12(candidate_12, optimizer_12, batch_12)
        if update_12 % 60 != 0:
            continue
        scores_12 = evaluate_12(
            candidate_12, validation_12,
            generation_max_new_tokens=cap_12,
        )
        history_12.append((rate_12, update_12, scores_12))
        key_12 = (scores_12["nll"], update_12, rate_12)
        if key_12 < best_key_12:
            best_key_12 = key_12
            winner_12 = deepcopy(candidate_12)
```

The history displays the loss used for selection along with exact match and EOS rates for the same validation predictions. Here exact match need not improve at every evaluation even as NLL falls. Teacher-forced NLL rewards probabilities assigned to the correct labels, while exact match requires every greedy decision and EOS to be correct on a generated sequence. Only the NLL enters the selection key.

Now freeze the winning choice and call `evaluate` once on its test split. Report NLL per evaluated token and counts per generated sequence. The two denominators answer different questions.

In [24]

```
print("    LR  Update    Val NLL   Val exact   Val EOS")
for rate_12, update_12, scores_12 in sorted(
    history_12, key=lambda row: row[:2]
):
    print(
        f'{rate_12:6.3f} {update_12:7d} {scores_12["nll"]:10.6f} '
        f'{scores_12["exact_match"]:10.2%} '
        f'{scores_12["termination_rate"]:9.2%}'
    )

nll_12, selected_update_12, selected_rate_12 = best_key_12
print(f"\nSelected LR {selected_rate_12}, update {selected_update_12}")
print(f"Validation NLL {nll_12:.6f}")
test_metrics_12 = evaluate_12(
    winner_12, test_12, generation_max_new_tokens=cap_12
)
sequences_12 = test_metrics_12["sequences"]
exact_12 = round(test_metrics_12["exact_match"] * sequences_12)
ended_12 = round(test_metrics_12["termination_rate"] * sequences_12)
print(f'Test tokens {test_metrics_12["tokens"]}')
print(f'Test NLL {test_metrics_12["nll"]:.6f}')
print(f"Test exact match {exact_12}/{sequences_12}")
print(f"Test termination {ended_12}/{sequences_12}")
```

Output

```
    LR  Update    Val NLL   Val exact   Val EOS
 0.001       0   3.024102      0.00%    14.06%
 0.001      60   1.087083      1.56%    96.88%
 0.001     120   0.583314     31.25%   100.00%
 0.001     180   0.418732     37.50%   100.00%
 0.001     240   0.323457     51.56%   100.00%
 0.004       0   3.024102      0.00%    14.06%
 0.004      60   0.656598     20.31%    98.44%
 0.004     120   0.301420     59.38%   100.00%
 0.004     180   0.256519     64.06%   100.00%
 0.004     240   0.252429     57.81%   100.00%

Selected LR 0.004, update 240
Validation NLL 0.252429
Test tokens 352
Test NLL 0.257212
Test exact match 42/64
Test termination 63/64
```

The selection rule chooses learning rate 0.004 at update 240, whose validation NLL is 0.252429. For that learning rate, validation exact match is 64.06% at update 180 and 57.81% at update 240. The specified rule still chooses update 240 because its NLL is lower: 0.252429 versus 0.256519. These metrics need not rank checkpoints identically.

Only after freezing that choice do we evaluate the test split: NLL 0.257212 over 352 labels, 42/64 exact answers, and 63/64 answers ending by the cap. The generation cap is 10 for every comparison. These are results for this smaller selection experiment, not the chapter's 2,000-update model. Choosing another model after seeing these test results would reuse the test set for selection.

13. Detect information leakage (optional experiment)

In the shifted input `BOS blue green red`, the first label blue occurs at the second input position. Allowing future keys reveals the first three color labels, but EOS has no revealing input. Keeping the shift is therefore insufficient without the causal mask.

Compare two fresh models with the same seed, data, minibatches, and optimizer settings. The only difference is the target attention mask. A wrapper calls the original mask builder to preserve its validation and source mask, then keeps target key validity while removing the triangle. Both imported bindings are patched together. Context managers restore them even if the experiment raises an exception.

Use 128 training and 32 validation sequences of lengths 3 and 4, width 64, four heads, FFN width 128, two layers per stack, and default module initialization from seed 2026. Train each model for 1,000 updates with Adam rate 0.001 and minibatches of 32. Use generation cap six. The test split is held aside; these are validation diagnostics.

In [25]

```
from examples.ch06_training.data import make_splits, words
from examples.ch06_training.training import exact_matches
original_masks = architecture.build_masks
assert decoding.build_masks is original_masks

def future_keys(source_valid, target_valid):
    source_allowed, _ = original_masks(source_valid, target_valid)
    return source_allowed, target_valid[:, None, None, :]

validity = torch.tensor([[True, True, True, False]])
print("Causal first-query keys:", original_masks(validity, validity)[1][0,0,0].int().tolist())
print("Future first-query keys:", future_keys(validity, validity)[1][0,0,0].int().tolist())
```

Output

```
Causal first-query keys: [1, 0, 0, 0]
Future first-query keys: [1, 1, 1, 0]
```

Measure three behaviors. Whole-reference NLL lets each query see every key its mask allows. Prefix-only NLL truncates the input before every reference label, supplying the true past but withholding future inputs. Free generation uses only the model's own previous choices.

For the causal model, whole-reference and prefix-only NLL should agree within numerical tolerance. For the noncausal model, their gap reveals dependence on future reference inputs. Both free-generation comparisons use `cached=False`: adding a token can change earlier states in a noncausal decoder, invalidating old K/V. The normal evaluator uses a cache and is therefore inappropriate for this altered model.

In [26]

```
@torch.no_grad()
def leakage_scores(model, validation, cap=6):
    model.eval()
    total_loss = prefix_loss = 0.
    tokens = matches = ended = 0
    examples = []
    for start in range(0, len(validation), 16):
        batch = collate(validation[start:start+16])
        total, count = token_loss(model(batch.source, batch.decoder_input), batch.target)
        total_loss += total.item()
        tokens += count.item()
        memory, allowed = decoding.encode_source(model, batch.source)
        for position in range(batch.target.shape[1]):
            active = batch.target[:, position] != PAD
            logits = decoding.decode_prefix(
                model, batch.decoder_input[active, :position+1],
                memory[active], allowed[active])[:, -1]
            partial, _ = token_loss(logits, batch.target[active, position])
            prefix_loss += partial.item()
        generated = decoding.greedy_decode(model, batch.source,
                                           max_new_tokens=cap, cached=False)
        matches += exact_matches(generated, batch.target)
        ended += generated.eq(EOS).any(1).sum().item()
        for source, target, result in zip(batch.source, batch.target, generated):
            examples.append(dict(source=words(source),
                reference=words(target, True), generated=words(result, True)))
    return dict(full_nll=total_loss/tokens, prefix_nll=prefix_loss/tokens,
                exact=matches, terminated=ended, tokens=tokens, examples=examples)
```

In [27]

```
def fit_mask_experiment():
    torch.manual_seed(2026)
    sampler = random.Random(2026)
    train, validation, _ = make_splits(seed=2026, n_train=128,
        n_validation=32, n_test=32, min_length=3, max_length=4)
    model = Transformer(9, 9, d_model=64, n_heads=4, d_ff=128,
        n_encoder_layers=2, n_decoder_layers=2, dropout=0.0)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    for _ in range(1000):
        train_step(model, optimizer, collate(sampler.sample(train, 32)))
    return leakage_scores(model, validation)

causal_run = fit_mask_experiment()
with patch.object(architecture, "build_masks", future_keys),      patch.object(decoding, "build_masks",
     future_keys):
    broken_run = fit_mask_experiment()
assert architecture.build_masks is decoding.build_masks is original_masks
assert abs(causal_run["full_nll"]-causal_run["prefix_nll"]) < 1e-6
print("Mask         Whole NLL  Prefix NLL  Exact  EOS")
for name, row in [("Causal", causal_run), ("Future keys", broken_run)]:
    print(f"{name:11s} {row['full_nll']:9.4f} {row['prefix_nll']:11.4f} "
          f"{row['exact']:3d}/32 {row['terminated']:3d}/32")
for base, broken in zip(causal_run["examples"][:3], broken_run["examples"]):
    print("Source:", base["source"])
    print("Reference:", base["reference"])
    print("Causal:", base["generated"], "| Future:", broken["generated"])
print("Both original mask bindings restored.")
```

Output

```
Mask         Whole NLL  Prefix NLL  Exact  EOS
Causal         0.8531      0.8531  13/32  32/32
Future keys    0.8460      7.1246   0/32  32/32
Source: green green white green
Reference: green white green green <eos>
Causal: green white green <eos> | Future: <eos>
Source: black white red red
Reference: red red white black <eos>
Causal: red red white black <eos> | Future: <eos>
Source: blue red white red
Reference: red white red blue <eos>
Causal: red white blue <eos> | Future: <eos>
Both original mask bindings restored.
```

The measured comparison is:

Mask	Whole-reference NLL	Prefix-only NLL	Exact answers	Ended by cap
Causal	0.8531	0.8531	13/32	32/32
Future keys allowed	0.8460	7.1246	0/32	32/32

Both losses average the same 144 reference labels. The causal NLLs agree because truncation removes only keys already forbidden by the triangle. The noncausal NLL jumps when future inputs disappear, despite the past reference tokens still being correct. Its slightly lower whole-reference loss therefore does not mean it is a better next-token predictor under the information available at generation time.

The noncausal model emits EOS immediately on each of the three displayed examples, including `black white red red`, whose answer should be `red red white black EOS`. It terminates without supplying the required colors. Both mask bindings are restored afterward; subsequent cache tests use the original causal model.

14. Break and repair a cache (optional experiment)

Compare incremental decoding with the full-prefix decoder using the same model, encoded source, and input tokens. The small model uses float64, two decoder layers, two attention heads, and head width 4. Evaluation mode makes dropout inactive. The shorter source row has one padded column. Both decoding paths read the same encoder memory with the same source mask, so neither attends to that padded column.

Use the shifted reference input as a predetermined prefix. At the final column, the shorter row has `PAD`. For this comparison feed a repeated `EOS` there in both paths because `CachedDecoder.step` rejects `PAD` inputs. The test compares logits on supplied prefixes, so no token selection or stopping rule can change the tokens being compared.

In [28]

```
model_14 = make_ch06_checks_model()
batch_14 = collate([(3, 4, 5, 6), (6, 4, 3)])
prefix_14 = batch_14.decoder_input.masked_fill(
    batch_14.decoder_input == PAD, EOS
)
memory_14, source_allowed_14 = decoding.encode_source(
    model_14, batch_14.source
)
print("Sources            ", batch_14.source.tolist())
print("Allowed source keys", source_allowed_14[:, 0, 0].int().tolist())
print("Supplied prefixes  ", prefix_14.tolist())
```

Output

```
Sources             [[3, 4, 5, 6], [6, 4, 3, 0]]
Allowed source keys [[1, 1, 1, 1], [1, 1, 1, 0]]
Supplied prefixes   [[1, 6, 5, 4, 3], [1, 3, 4, 6, 2]]
```

At each length t, pass just the new input column to the cache. The reference call sees the same first t columns together, so its last logit row is the prediction after consuming that same new token. Compare the full logit vectors for both examples. Each layer's self cache should have key and value shapes $[2, 2, t, 4]$ as its token axis grows.

For the broken variant, subclass `CachedDecoder` and reset its position counter before each call. The layer caches still grow, but every new input receives the positional vector for index 0. Run the repaired comparison with a fresh canonical decoder. A fresh cache recomputes the earlier keys and values with the correct positions.

In [29]

```
class ZeroPositionDecoder(decoding.CachedDecoder):
    def step(self, token):
        self.position = 0
        return super().step(token)

def compare_prefixes_14(cache_class):
    cache = cache_class(model_14, memory_14, source_allowed_14)
    rows = []
    for length in range(1, prefix_14.shape[1] + 1):
        token = prefix_14[:, length - 1:length]
        incremental = cache.step(token)
        full = decoding.decode_prefix(
            model_14, prefix_14[:, :length], memory_14, source_allowed_14
        )[:, -1]
        for key, value in cache.self_kv:
            assert key.shape == value.shape == (2, 2, length, 4)
        max_error = (incremental - full).abs().max().item()
        close = torch.allclose(incremental, full, atol=1e-12, rtol=1e-12)
        rows.append((max_error, close))
    return rows
```

In [30]

```
clean_14 = compare_prefixes_14(decoding.CachedDecoder)
broken_14 = compare_prefixes_14(ZeroPositionDecoder)
repaired_14 = compare_prefixes_14(decoding.CachedDecoder)

assert all(close for _, close in clean_14 + repaired_14)
first_failure_14 = next(
    length for length, (_, close) in enumerate(broken_14, 1)
    if not close
)
assert first_failure_14 == 2
print("Prefix length   Baseline error   Zero position   Repaired error")
for length, rows in enumerate(zip(clean_14, broken_14, repaired_14), 1):
    baseline, broken, repaired = (row[0] for row in rows)
    print(f"{length:13d}   {baseline:14.3e}   {broken:13.3e}   "
          f"{repaired:14.3e}")
print(f"First failing prefix length  {first_failure_14}")
```

Output

```
Prefix length   Baseline error   Zero position   Repaired error
            1        0.000e+00       0.000e+00        0.000e+00
            2        0.000e+00       1.632e-01        0.000e+00
            3        0.000e+00       3.174e-01        0.000e+00
            4        2.220e-16       1.934e-01        2.220e-16
            5        3.053e-16       9.305e-02        3.053e-16
First failing prefix length  2
```

The first input belongs at position index 0 in both paths. The bug becomes visible at length 2, when the new token belongs at index 1 but receives index 0 again. The wrong positional vector changes its representation before projection to queries, keys, and values. The output logits then differ, and the appended incorrect keys and values persist into later steps. A first-token check cannot exercise the position advance.

15. Time equal work (optional experiment)

Use one fixed CPU source and a five-token, nonpadding target prefix. The full-prefix decoder recomputes all five positions; the cache consumes the first four in advance and then processes the fifth. Check that their final-input logits agree before timing.

The four phases have separate boundaries. Source encoding produces memory and its mask. Cached target prefill constructs static cross-KV and consumes the first four inputs in successive one-token steps. The two later-token measurements process the fifth input, either from the full prefix or from its cache. Each cached repetition starts with a fresh prefilled cache, constructed outside that measured step.

In [31]

```
model_15 = make_ch06_checks_model()
batch_15 = collate([(3, 4, 5, 6)])
source_15, prefix_15 = batch_15.source, batch_15.decoder_input
assert prefix_15.shape == (1, 5) and not prefix_15.eq(PAD).any()

memory_15, allowed_15 = decoding.encode_source(model_15, source_15)
cache_15 = decoding.CachedDecoder(model_15, memory_15, allowed_15)
for column_15 in prefix_15.split(1, dim=1):
    last_incremental_15 = cache_15.step(column_15)
full_15 = decoding.decode_prefix(
    model_15, prefix_15, memory_15, allowed_15
)
torch.testing.assert_close(
    last_incremental_15, full_15[:, -1], atol=1e-12, rtol=1e-12
)
print("Source IDs:", source_15.tolist())
print("Fixed target input IDs:", prefix_15.tolist())
print("Final-input logits agree to float64 tolerance.")
```

Output

```
Source IDs: [[3, 4, 5, 6]]
Fixed target input IDs: [[1, 6, 5, 4, 3]]
Final-input logits agree to float64 tolerance.
```

In [32]

```
previous_threads_15 = torch.get_num_threads()
try:
    torch.set_num_threads(1)
    timings_15 = [
        decoding.benchmark(model_15, source_15, prefix_15, repeats=25)
        for _ in range(2)
    ]
finally:
    torch.set_num_threads(previous_threads_15)

timing_columns_15 = (
    ("Source encoding", "source_encode_ms"),
    ("Cached target prefill", "cached_target_prefill_ms"),
    ("Full-prefix decoder", "full_prefix_next_token_ms"),
    ("Cached final input", "cached_next_token_ms"),
)
print(f"CPU   {next(model_15.parameters()).dtype}   "
      f"{timings_15[0]['threads']} thread   25 repeats per run")
print("Median milliseconds per phase        Run 1     Run 2")
for label, key in timing_columns_15:
    left, right = (result[key] for result in timings_15)
    assert all(math.isfinite(ms) and ms > 0 for ms in (left, right))
    print(f"{label:32s}  {left:7.3f}   {right:7.3f}")
```

Output

```
CPU   torch.float64   1 thread   25 repeats per run
Median milliseconds per phase        Run 1     Run 2
Source encoding                     0.143     0.150
Cached target prefill               0.468     0.476
Full-prefix decoder                 0.238     0.264
Cached final input                  0.119     0.120
```

A fixed prefix ensures both paths compute the same requested prediction. An early EOS could otherwise shorten one run; different continuations could change its attention work. Including source encoding in only one path would also compare different work. Python overhead and concatenation copies are substantial in a model this small. The timings describe this CPU, model, and prefix length.

In the saved float64 timing check (one CPU thread, 25 repeats per run), the phase medians were:

Phase	Run 1 (ms)	Run 2 (ms)
Source encoding	0.143	0.150
Cached target prefill, including cross-KV setup	0.468	0.476
Full-prefix final-input calculation	0.238	0.264
Cached final-input calculation	0.119	0.120

Compare only the last two rows as later-token timings. Selection, data loading, and text conversion are excluded; these are not end-to-end request latencies or forecasts of GPU speed.